

PRACTICAL SHEET 4 (Aryan Kushwaha – 245703428)

Q1: // Singly Linked List (Insert, Delete, Traverse)

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void insertFront(Node*& head, int val) {
    Node* newNode = new Node{val, head};
    head = newNode;
}

void insertEnd(Node*& head, int val) {
    Node* newNode = new Node{val, nullptr};
    if (!head) {
        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
}

void insertPos(Node*& head, int val, int pos) {
    if (pos == 0) {
        insertFront(head, val);
        return;
    }
    Node* temp = head;
    for (int i = 0; i < pos - 1 && temp; ++i) {
        temp = temp->next;
    }
    if (!temp) return;
    Node* newNode = new Node{val, temp->next};
    temp->next = newNode;
}

void deleteFront(Node*& head) {
    if (!head) return;
    Node* temp = head;
    head = head->next;
    delete temp;
}

void deleteEnd(Node*& head) {
    if (!head) return;
    if (!head->next) {
        delete head;
        head = nullptr;
    }
```

```

        return;
    }
    Node* temp = head;
    while (temp->next->next) temp = temp->next;
    delete temp->next;
    temp->next = nullptr;
}

void deletePos(Node*& head, int pos) {
    if (pos == 0) {
        deleteFront(head);
        return;
    }
    Node* temp = head;
    for (int i = 0; i < pos - 1 && temp && temp->next; ++i) {
        temp = temp->next;
    }
    if (!temp || !temp->next) return;
    Node* toDelete = temp->next;
    temp->next = temp->next->next;
    delete toDelete;
}

void printList(Node* head) {
    while (head) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    Node* head = nullptr;
    insertEnd(head, 10);
    insertFront(head, 5);
    insertEnd(head, 30);
    insertPos(head, 20, 2);

    cout << "After Insertions: ";
    printList(head);

    deleteFront(head);
    deleteEnd(head);
    deletePos(head, 0);

    cout << "After Deletions: ";
    printList(head);

    return 0;
}

```

Output:

```
After Insertions: 5 -> 10 -> 20 -> 30 -> NULL
After Deletions: 20 -> NULL
```

Q2: // Implement a sequence using a linked list

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void insertFront(Node*& head, int val) {
    Node* newNode = new Node;
    newNode->data = val;
    newNode->next = head;
    head = newNode;
}

void insertEnd(Node*& head, int val) {
    Node* newNode = new Node;
    newNode->data = val;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = newNode;
        return;
    }

    Node* temp = head;
    while (temp->next != nullptr) {
        temp = temp->next;
    }
    temp->next = newNode;
}

void reverseList(Node*& head) {
    Node* prev = nullptr;
    Node* current = head;
    Node* next = nullptr;

    while (current != nullptr) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    head = prev;
}
```

```

void printList(Node* head) {
    while (head != nullptr) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    Node* head = nullptr;

    insertEnd(head, 10);
    insertEnd(head, 20);
    insertFront(head, 5);
    insertEnd(head, 30);

    cout << "Original List: ";
    printList(head);

    reverseList(head);

    cout << "Reversed List: ";
    printList(head);

    return 0;
}

```

Output:

```

Original List: 5 -> 10 -> 20 -> 30 -> NULL
Reversed List: 30 -> 20 -> 10 -> 5 -> NULL

```

```

Q3: // Find the Longest Increasing Subsequence (LIS)

#include <iostream>
using namespace std;

int longestIncreasingSubsequence(int arr[], int n) {
    if (n == 0) return 0;

    int* dp = new int[n];
    int maxLIS = 1;

    for (int i = 0; i < n; ++i) {
        dp[i] = 1;
        for (int j = 0; j < i; ++j) {
            if (arr[i] > arr[j] && dp[i] < dp[j] + 1) {
                dp[i] = dp[j] + 1;
            }
        }
        if (dp[i] > maxLIS) {

```

```

        maxLIS = dp[i];
    }
}

delete[] dp;
return maxLIS;
}

int main() {
    int arr[] = {10, 22, 9, 33, 21, 50, 41, 60, 80};
    int n = sizeof(arr) / sizeof(arr[0]);

    int result = longestIncreasingSubsequence(arr, n);

    cout << "Sequence: ";
    for (int i = 0; i < n; ++i) cout << arr[i] << " ";
    cout << "\nLength of LIS is: " << result << "\n";

    return 0;
}

```

Output:

```

Sequence: 10 22 9 33 21 50 41 60 80
Length of LIS is: 6

```

Q4: // Check whether a sequence is a palindrome

```

#include <iostream>
using namespace std;

bool isPalindrome(int arr[], int size) {
    int left = 0;
    int right = size - 1;

    while (left < right) {
        if (arr[left] != arr[right]) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}

int main() {
    int seq1[] = {1, 2, 3, 2, 1};
    int size1 = sizeof(seq1) / sizeof(seq1[0]);

    int seq2[] = {1, 2, 3, 4, 5};
    int size2 = sizeof(seq2) / sizeof(seq2[0]);

    cout << "Sequence 1 is palindrome: "
         << (isPalindrome(seq1, size1) ? "Yes" : "No") << "\n";
}

```

```

    cout << "Sequence 2 is palindrome: "
          << (isPalindrome(seq2, size2) ? "Yes" : "No") << "\n";

    return 0;
}

```

Output:

```

Sequence 1 is palindrome: Yes
Sequence 2 is palindrome: No

```

Q5: // Merge two sorted sequences

```

#include <iostream>
using namespace std;

void mergeSorted(int arr1[], int size1, int arr2[], int size2, int result[]) {
    int i = 0, j = 0, k = 0;

    while (i < size1 && j < size2) {
        if (arr1[i] <= arr2[j]) {
            result[k++] = arr1[i++];
        } else {
            result[k++] = arr2[j++];
        }
    }

    while (i < size1) {
        result[k++] = arr1[i++];
    }

    while (j < size2) {
        result[k++] = arr2[j++];
    }
}

int main() {
    int arr1[] = {1, 3, 5, 7};
    int size1 = sizeof(arr1) / sizeof(arr1[0]);

    int arr2[] = {2, 4, 6, 8, 10};
    int size2 = sizeof(arr2) / sizeof(arr2[0]);

    int resultSize = size1 + size2;
    int* result = new int[resultSize];

    mergeSorted(arr1, size1, arr2, size2, result);

    cout << "Merged sorted sequence: ";
    for (int i = 0; i < resultSize; ++i) {
        cout << result[i] << " ";
    }
}

```

```

}
cout << "\n";

delete[] result;
return 0;
}

```

Output:

```
Merged sorted sequence: 1 2 3 4 5 6 7 8 10
```

Q6: // Implement a sequence using an array (Insert, Delete, Length)

```

#include <iostream>
using namespace std;

void insertElement(int arr[], int& size, int pos, int val, int capacity) {
    if (pos < 0 || pos > size || size >= capacity) {
        cout << "Invalid position or array is full.\n";
        return;
    }
    for (int i = size; i > pos; --i) {
        arr[i] = arr[i - 1];
    }
    arr[pos] = val;
    size++;
}

void deleteElement(int arr[], int& size, int pos) {
    if (pos < 0 || pos >= size) {
        cout << "Invalid position.\n";
        return;
    }
    for (int i = pos; i < size - 1; ++i) {
        arr[i] = arr[i + 1];
    }
    size--;
}

int getLength(int size) {
    return size;
}

int main() {
    int arr[100];
    int size = 0;
    int capacity = 100;

    insertElement(arr, size, 0, 10, capacity);
    insertElement(arr, size, 1, 20, capacity);
    insertElement(arr, size, 1, 15, capacity);
}

```

```

    cout << "Array after insertions: ";
    for (int i = 0; i < size; ++i) cout << arr[i] << " ";
    cout << "\nLength: " << getLength(size) << "\n";

    deleteElement(arr, size, 1);

    cout << "Array after deletion at index 1: ";
    for (int i = 0; i < size; ++i) cout << arr[i] << " ";
    cout << "\nLength: " << getLength(size) << "\n";

    return 0;
}

```

Output:

```

Array after insertions: 10 15 20
Length: 3
Array after deletion at index 1: 10 20
Length: 2

```

Q7: // Doubly Linked List (Insert, Delete, Traverse)

```

#include <iostream>
using namespace std;

struct DNode {
    int data;
    DNode* prev;
    DNode* next;
};

void insertEnd(DNode*& head, int val) {
    DNode* newNode = new DNode{val, nullptr, nullptr};
    if (!head) {
        head = newNode;
        return;
    }
    DNode* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}

void deleteNode(DNode*& head, int val) {
    if (!head) return;
    DNode* temp = head;

    while (temp && temp->data != val) {
        temp = temp->next;
    }

    if (!temp) return;

```

```

        if (temp->prev) temp->prev->next = temp->next;
        else head = temp->next;

        if (temp->next) temp->next->prev = temp->prev;

        delete temp;
    }

void printList(DNode* head) {
    while (head) {
        cout << head->data << " <-> ";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    DNode* head = nullptr;
    insertEnd(head, 10);
    insertEnd(head, 20);
    insertEnd(head, 30);

    cout << "Doubly Linked List: ";
    printList(head);

    deleteNode(head, 20);

    cout << "After deleting 20: ";
    printList(head);

    return 0;
}

```

Output:

```

Doubly Linked List: 10 <-> 20 <-> 30 <-> NULL
After deleting 20: 10 <-> 30 <-> NULL

```

```

Q8: // Concatenate two linked lists

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void insertEnd(Node*& head, int val) {
    Node* newNode = new Node{val, nullptr};
    if (!head) {

```

```

        head = newNode;
        return;
    }
    Node* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
}

Node* concatenate(Node* head1, Node* head2) {
    if (!head1) return head2;
    Node* temp = head1;
    while (temp->next) {
        temp = temp->next;
    }
    temp->next = head2;
    return head1;
}

void printList(Node* head) {
    while (head) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    Node* list1 = nullptr;
    Node* list2 = nullptr;

    insertEnd(list1, 1);
    insertEnd(list1, 2);

    insertEnd(list2, 3);
    insertEnd(list2, 4);

    Node* result = concatenate(list1, list2);

    cout << "Concatenated List: ";
    printList(result);

    return 0;
}

```

Output:

```
Concatenated List: 1 -> 2 -> 3 -> 4 -> NULL
```

Q9: // Split a circular linked list into two halves

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void insertEnd(Node*& head, int val) {
    Node* newNode = new Node{val, nullptr};
    if (!head) {
        head = newNode;
        newNode->next = head;
        return;
    }
    Node* temp = head;
    while (temp->next != head) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->next = head;
}

void splitCircularList(Node* head, Node*& head1, Node*& head2) {
    if (!head) return;

    Node* slow = head;
    Node* fast = head;

    while (fast->next != head && fast->next->next != head) {
        fast = fast->next->next;
        slow = slow->next;
    }

    if (fast->next->next == head) {
        fast = fast->next;
    }

    head1 = head;
    if (head->next != head) {
        head2 = slow->next;
    } else {
        head2 = nullptr;
    }

    fast->next = head2;
    slow->next = head1;
}

void printCircularList(Node* head) {
    if (!head) return;
    Node* temp = head;
```

```

do {
    cout << temp->data << " -> ";
    temp = temp->next;
} while (temp != head);
cout << "(HEAD: " << head->data << ")\n";
}

int main() {
    Node* head = nullptr;
    Node *head1 = nullptr, *head2 = nullptr;

    insertEnd(head, 1);
    insertEnd(head, 2);
    insertEnd(head, 3);
    insertEnd(head, 4);
    insertEnd(head, 5);

    cout << "Original Circular List: ";
    printCircularList(head);

    splitCircularList(head, head1, head2);

    cout << "First Half: ";
    printCircularList(head1);

    cout << "Second Half: ";
    printCircularList(head2);

    return 0;
}

```

Output:

```

Original Circular List: 1 -> 2 -> 3 -> 4 -> 5 -> (HEAD: 1)
First Half: 1 -> 2 -> 3 -> (HEAD: 1)
Second Half: 4 -> 5 -> (HEAD: 4)

```

Q10: // Detect and remove a cycle in a linked list

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

void insertEnd(Node*& head, int val) {
    Node* newNode = new Node{val, nullptr};
    if (!head) {
        head = newNode;
        return;
    }
}

```

```

    }
    Node* temp = head;
    while (temp->next) temp = temp->next;
    temp->next = newNode;
}

void detectAndRemoveCycle(Node* head) {
    if (!head || !head->next) return;

    Node* slow = head;
    Node* fast = head;
    bool hasCycle = false;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            hasCycle = true;
            break;
        }
    }

    if (hasCycle) {
        slow = head;
        if (slow == fast) {
            while (fast->next != slow) {
                fast = fast->next;
            }
        } else {
            while (slow->next != fast->next) {
                slow = slow->next;
                fast = fast->next;
            }
        }
        fast->next = nullptr;
        cout << "Cycle detected and removed.\n";
    } else {
        cout << "No cycle detected.\n";
    }
}

void printList(Node* head) {
    while (head) {
        cout << head->data << " -> ";
        head = head->next;
    }
    cout << "NULL\n";
}

int main() {
    Node* head = nullptr;
    insertEnd(head, 1);
    insertEnd(head, 2);
    insertEnd(head, 3);
}

```

```

insertEnd(head, 4);
insertEnd(head, 5);

head->next->next->next->next->next = head->next->next;

detectAndRemoveCycle(head);

cout << "List after cycle removal: ";
printList(head);

return 0;
}

```

Output:

```

Cycle detected and removed.
List after cycle removal: 1 -> 2 -> 3 -> 4 -> 5 -> NULL

```

Q11: // Hash table using division method and linear probing

```

#include <iostream>
using namespace std;

const int TABLE_SIZE = 10;

void insert(int table[], int key) {
    int index = key % TABLE_SIZE;
    int i = 0;

    while (table[(index + i) % TABLE_SIZE] != -1 && i < TABLE_SIZE) {
        i++;
    }

    if (i == TABLE_SIZE) {
        cout << "Table is full.\n";
        return;
    }

    table[(index + i) % TABLE_SIZE] = key;
}

void printTable(int table[]) {
    for (int i = 0; i < TABLE_SIZE; ++i) {
        if (table[i] != -1) {
            cout << "Index " << i << ": " << table[i] << "\n";
        } else {
            cout << "Index " << i << ": Empty\n";
        }
    }
}

```

```

int main() {
    int table[TABLE_SIZE];
    for (int i = 0; i < TABLE_SIZE; ++i) {
        table[i] = -1;
    }

    insert(table, 15);
    insert(table, 25);
    insert(table, 35);
    insert(table, 12);
    insert(table, 22);

    printTable(table);

    return 0;
}

```

Output:

```

Index 0: Empty
Index 1: Empty
Index 2: 12
Index 3: 22
Index 4: Empty
Index 5: 15
Index 6: 25
Index 7: 35
Index 8: Empty
Index 9: Empty

```

```

Q12: // Hash table with quadratic probing

#include <iostream>
using namespace std;

const int TABLE_SIZE = 11;

void insert(int table[], int key) {
    int index = key % TABLE_SIZE;
    int i = 0;

    while (table[(index + i * i) % TABLE_SIZE] != -1 && i < TABLE_SIZE) {
        i++;
    }

    if (i == TABLE_SIZE) {
        cout << "Cannot insert, table full or probing failed.\n";
        return;
    }

    table[(index + i * i) % TABLE_SIZE] = key;
}

```

```

void printTable(int table[]) {
    for (int i = 0; i < TABLE_SIZE; ++i) {
        if (table[i] != -1) {
            cout << "Index " << i << ": " << table[i] << "\n";
        } else {
            cout << "Index " << i << ": Empty\n";
        }
    }
}

int main() {
    int table[TABLE_SIZE];
    for (int i = 0; i < TABLE_SIZE; ++i) {
        table[i] = -1;
    }

    insert(table, 10);
    insert(table, 21);
    insert(table, 32);
    insert(table, 43);

    printTable(table);

    return 0;
}

```

Output:

```

Index 0: 21
Index 1: Empty
Index 2: Empty
Index 3: 32
Index 4: Empty
Index 5: Empty
Index 6: Empty
Index 7: Empty
Index 8: 43
Index 9: Empty
Index 10: 10

```

Q13: // Hash table using chaining with linked lists

```

#include <iostream>
using namespace std;

const int TABLE_SIZE = 7;

struct Node {
    int data;
    Node* next;
};

```

```

void insert(Node* table[], int key) {
    int index = key % TABLE_SIZE;
    Node* newNode = new Node{key, table[index]};
    table[index] = newNode;
}

void printTable(Node* table[]) {
    for (int i = 0; i < TABLE_SIZE; ++i) {
        cout << "Index " << i << ": ";
        Node* temp = table[i];
        while (temp) {
            cout << temp->data << " -> ";
            temp = temp->next;
        }
        cout << "NULL\n";
    }
}

int main() {
    Node* table[TABLE_SIZE] = {nullptr};

    insert(table, 10);
    insert(table, 17);
    insert(table, 24);
    insert(table, 5);
    insert(table, 12);

    printTable(table);

    return 0;
}

```

Output:

```

Index 0: NULL
Index 1: NULL
Index 2: NULL
Index 3: 24 -> 17 -> 10 -> NULL
Index 4: NULL
Index 5: 12 -> 5 -> NULL
Index 6: NULL

```

Q14: // Hash table for strings with search function

```

#include <iostream>
#include <string>
using namespace std;

const int TABLE_SIZE = 10;

struct StringNode {
    string key;

```

```

    StringNode* next;
};

int hashFunction(const string& str) {
    int hash = 0;
    for (int i = 0; i < str.length(); ++i) {
        hash = (hash * 31 + str[i]) % TABLE_SIZE;
    }
    return hash;
}

void insert(StringNode* table[], const string& key) {
    int index = hashFunction(key);
    StringNode* newNode = new StringNode{key, table[index]};
    table[index] = newNode;
}

bool search(StringNode* table[], const string& key) {
    int index = hashFunction(key);
    StringNode* temp = table[index];
    while (temp) {
        if (temp->key == key) {
            return true;
        }
        temp = temp->next;
    }
    return false;
}

int main() {
    StringNode* table[TABLE_SIZE] = {nullptr};

    insert(table, "apple");
    insert(table, "banana");
    insert(table, "orange");
    insert(table, "grape");

    string searchKey = "banana";
    if (search(table, searchKey)) {
        cout << searchKey << " found in the hash table.\n";
    } else {
        cout << searchKey << " not found.\n";
    }

    searchKey = "mango";
    if (search(table, searchKey)) {
        cout << searchKey << " found in the hash table.\n";
    } else {
        cout << searchKey << " not found.\n";
    }

    return 0;
}

```

Output:

```
banana found in the hash table.  
mango not found.
```

Q15: // Hash function for student records (Insert, Search, Delete)

```
#include <iostream>
#include <string>
using namespace std;

const int TABLE_SIZE = 10;

enum Status { EMPTY, OCCUPIED, DELETED };

struct Student {
    int roll;
    string name;
    Status status;
};

int hashFunc(int roll) {
    return roll % TABLE_SIZE;
}

void insert(Student table[], int roll, const string& name) {
    int index = hashFunc(roll);
    int i = 0;

    while (table[(index + i) % TABLE_SIZE].status == OCCUPIED && i < TABLE_SIZE) {
        if (table[(index + i) % TABLE_SIZE].roll == roll) {
            cout << "Roll number already exists.\n";
            return;
        }
        i++;
    }

    if (i == TABLE_SIZE) {
        cout << "Hash table is full.\n";
        return;
    }

    int finalIndex = (index + i) % TABLE_SIZE;
    table[finalIndex].roll = roll;
    table[finalIndex].name = name;
    table[finalIndex].status = OCCUPIED;
}

int search(Student table[], int roll) {
    int index = hashFunc(roll);
    int i = 0;
```

```

    while (table[(index + i) % TABLE_SIZE].status != EMPTY && i < TABLE_SIZE) {
        if (table[(index + i) % TABLE_SIZE].status == OCCUPIED && table[(index + i) %
TABLE_SIZE].roll == roll) {
            return (index + i) % TABLE_SIZE;
        }
        i++;
    }
    return -1;
}

void deleteStudent(Student table[], int roll) {
    int index = search(table, roll);
    if (index != -1) {
        table[index].status = DELETED;
        cout << "Student with roll " << roll << " deleted.\n";
    } else {
        cout << "Student not found.\n";
    }
}

int main() {
    Student table[TABLE_SIZE];
    for (int i = 0; i < TABLE_SIZE; ++i) {
        table[i].status = EMPTY;
    }

    insert(table, 101, "Alice");
    insert(table, 111, "Bob");
    insert(table, 121, "Charlie");

    int idx = search(table, 111);
    if (idx != -1) cout << "Found: " << table[idx].name << "\n";
    else cout << "Not found\n";

    deleteStudent(table, 111);

    idx = search(table, 111);
    if (idx != -1) cout << "Found: " << table[idx].name << "\n";
    else cout << "Not found\n";

    return 0;
}

```

Output:

```

Found: Bob
Student with roll 111 deleted.
Not found

```

Q16: // Primary clustering in linear probing solved using double hashing

```
#include <iostream>
using namespace std;

const int TABLE_SIZE = 11;
const int PRIME = 7;

void insertLinear(int table[], int key) {
    int index = key % TABLE_SIZE;
    int i = 0;
    while (table[(index + i) % TABLE_SIZE] != -1 && i < TABLE_SIZE) i++;
    if (i < TABLE_SIZE) table[(index + i) % TABLE_SIZE] = key;
}

void insertDouble(int table[], int key) {
    int h1 = key % TABLE_SIZE;
    int h2 = PRIME - (key % PRIME);
    int i = 0;
    while (table[(h1 + i * h2) % TABLE_SIZE] != -1 && i < TABLE_SIZE) i++;
    if (i < TABLE_SIZE) table[(h1 + i * h2) % TABLE_SIZE] = key;
}

void printTable(int table[]) {
    for (int i = 0; i < TABLE_SIZE; ++i) {
        if (table[i] != -1) cout << table[i] << " ";
        else cout << "- ";
    }
    cout << "\n";
}

int main() {
    int linearTable[TABLE_SIZE];
    int doubleTable[TABLE_SIZE];

    for (int i = 0; i < TABLE_SIZE; ++i) {
        linearTable[i] = -1;
        doubleTable[i] = -1;
    }

    int keys[] = {10, 21, 32, 43, 54};
    int numKeys = sizeof(keys) / sizeof(keys[0]);

    for (int i = 0; i < numKeys; ++i) {
        insertLinear(linearTable, keys[i]);
        insertDouble(doubleTable, keys[i]);
    }

    cout << "Linear Probing (Primary Clustering visible around index 10-3):\n";
    printTable(linearTable);

    cout << "\nDouble Hashing (Keys uniformly spread):\n";
    printTable(doubleTable);
}
```

```
    return 0;  
}
```

Output:

```
Linear Probing (Primary Clustering visible around index 10-3):  
21 32 43 54 - - - - - 10  
  
Double Hashing (Keys uniformly spread):  
- 54 32 - - 43 21 - - - 10
```